09

Part 9 · Performance Testing & Real-World Tuning

Measure, diagnose and tune real jobs, balancing speed and cost.

- Spark UI bottlenecks
- Benchmarking
- Reading metrics
- Diagnosing slow jobs
- Cost vs performance
- End-to-end pipelines

9.1 Identifying Bottlenecks in the Spark UI

★★★

IN ONE LINE

Follow jobs, then stages, then task metrics, then the plan to locate the real bottleneck.

Finding the bottleneck is a **funnel**: from the whole application, to the slowest job, to the slowest stage, to the pattern in its tasks, to the operator responsible. Do not tune before you know which pattern you have.

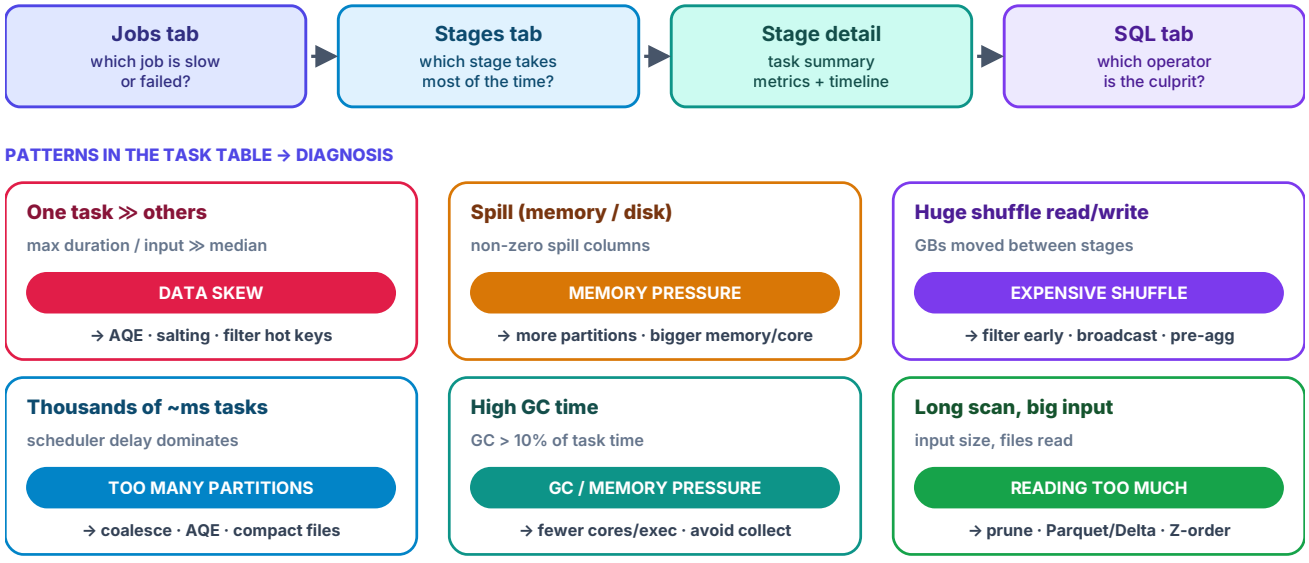


Figure 9.1 — The Spark UI funnel and six common patterns

Step	What to read	Question it answers
1 Jobs	Duration, failed jobs / tasks	Where does the time go overall?
2 Stages	Duration per stage, input / output / shuffle sizes	Which stage dominates?
3 Stage detail	Summary metrics (min / 25% / median / 75% / max), event timeline	Is it uniform, skewed, spilling, or tiny tasks?
4 SQL tab	Operator graph with rows and time	Which join / scan / aggregate causes it?
5 Executors	GC time, failed tasks, memory, lost executors	Is the cluster itself unhealthy?

INTERVIEW ANGLE

A crisp way to explain: *"I find the slowest stage, compare max to median task time to spot skew, check spill and shuffle sizes, then use the SQL tab to see which operator it is."* That single sentence covers most expected answers.

9.2 Benchmarking Jobs

★★☆

IN ONE LINE

Measure runtime and resources before and after a change under controlled, repeatable conditions.

Benchmarking means measuring runtime and resource use **before and after** a change, in a way that makes the comparison trustworthy. Without it, "optimisation" is guesswork.

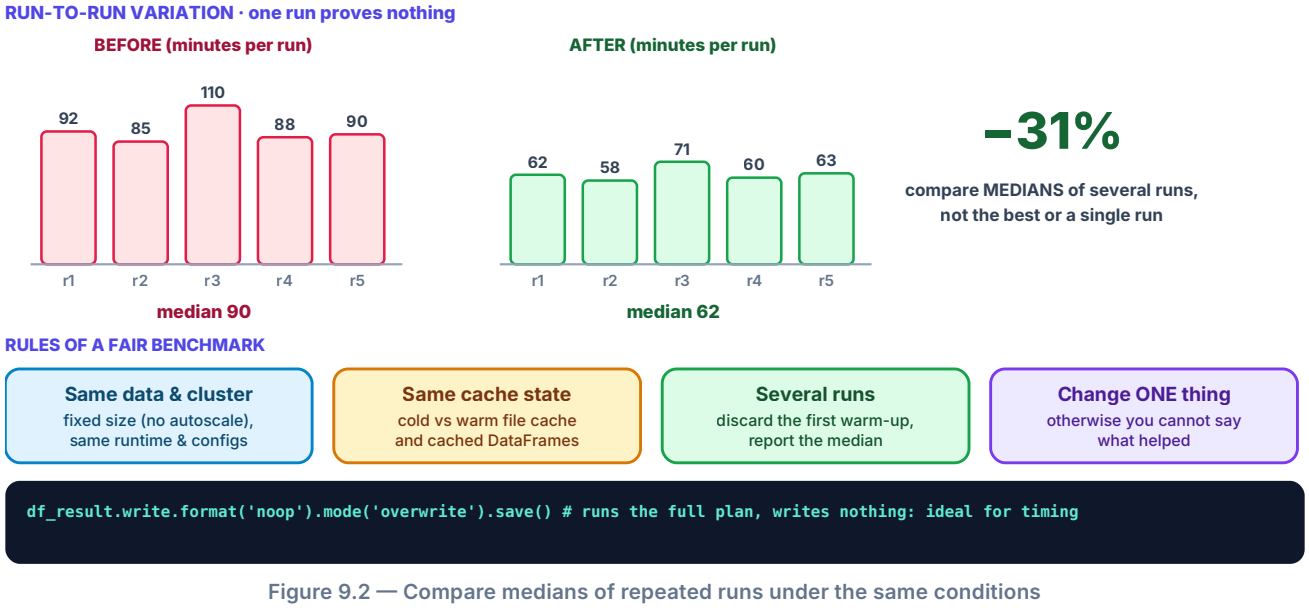


Figure 9.2 — Compare medians of repeated runs under the same conditions

Measure	Where to get it
Wall-clock runtime	Spark UI job / application duration, job run history
CPU / task time, shuffle and spill volumes	Stage metrics; compare **sums** as well as duration
Cost per run	Cluster DBUs / VM hours × price (7.4, 9.5)
Result correctness	Row counts, checksums, aggregates equal before vs after

A TINY TIMING HARNESS

```
import time

def timed(df, runs=3):
    times = []
    for _ in range(runs):
        t = time.time()
        df.write.format('noop').mode('overwrite').save()   # full execution, no output
        times.append(time.time() - t)
    return sorted(times)[len(times)//2]                    # median

print(timed(optimised_df))
```

COMMON TRAP

Cached data, warmed file cache, autoscaling, a noisy shared cluster and different input days all change timings. **Always re-validate that the optimised output equals the original**: a fast wrong answer is a failure.

9.3 Reading Spark Metrics

★★★

IN ONE LINE

Interpreting duration, GC, shuffle, spill and utilisation numbers in stage and executor views.

Spark records detailed **metrics per task** and summarises them per stage. A handful explain nearly every performance problem.

STAGE DETAIL · "Summary Metrics for Completed Tasks" (illustrative)

Metric	Min	25th	Median	75th	Max
Duration	0.8 s	1.0 s	1.1 s	1.3 s	4.2 min
GC Time	0 ms	0 ms	20 ms	40 ms	38 s
Input Size / Records	120 MB	125 MB	128 MB	128 MB	128 MB
Shuffle Read Size	40 MB	45 MB	50 MB	55 MB	3.1 GB
Spill (Memory)	0	0	0	0	8.4 GB
Spill (Disk)	0	0	0	0	1.9 GB

HOW TO READ IT

- **Max ≫ Median** — duration or shuffle read: SKEW (one hot partition)
- **Spill > 0** — task did not fit its data in execution memory
- **GC high** — object / memory pressure; large partitions or caching
- **All similar** — uniform tasks: look at the total work or the cluster size

Always compare the MAX column with the MEDIAN: averages hide stragglers.

Figure 9.3 — A task-summary table with a straggler, spill and a big shuffle read

Metric	Meaning	What it tells you
Duration	Task run time	Spread = balance; max vs median = skew
Scheduler delay / Task deserialisation	Time waiting for a slot / unpacking the task	High values with tiny tasks: too many partitions
GC time	Time paused in garbage collection	> ~10% of duration = memory pressure
Input size / records	Data read from the source	Volume per task; scan efficiency
Shuffle read / write	Data pulled from / written for shuffles	How much data moves; skew in read size
Spill (memory / disk)	Data spilled because it did not fit	Non-zero: more partitions or more memory per core
Peak execution memory	Max memory used by operators	Head-room against executor limits
Executor utilisation	Cores busy vs available (Executors tab)	Low = too few tasks, skew or idle waits

INTERVIEW ANGLE

The three-number habit: for the slowest stage note **median vs max task time**, **shuffle read/write size** and **spill**. If you can quote them, you sound like someone who has actually tuned a job.

9.4 Diagnosing Slow Jobs Systematically

★★★

IN ONE LINE

Locate the slowest stage, classify the pattern, apply one targeted fix, and verify.

Slow jobs have a few repeating causes. The skill is to **classify before you change anything**. Resist the urge to "just add more nodes" or random configs.

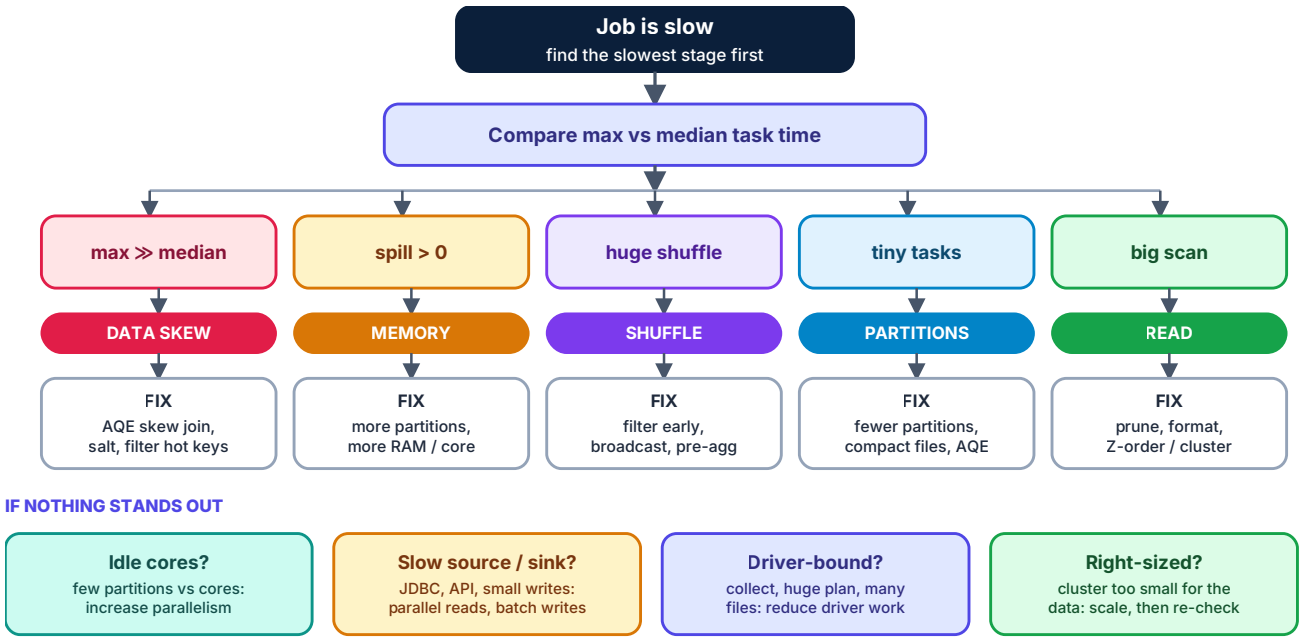


Figure 9.4 — Diagnosis flow: from symptom to cause to fix

A repeatable routine

- **Reproduce** with a known input and cluster.
- **Locate** the slowest stage (Spark UI).
- **Classify** with metrics (skew, spill, shuffle, tiny tasks, scan).
- **Hypothesise** one fix; apply it.
- **Measure** again; keep or revert.

Quick checks before deep work

- Is AQE on? Are stats/files sane?
- Did the input volume change recently?
- Is a join key duplicated or NULL-heavy?
- Did someone change the cluster / runtime?
- Is data being read from the right layout (pruning)?

EXAMPLE

"Stage 7 is 80% of runtime; max task 12 min, median 8 s; shuffle read max 6 GB vs median 40 MB." → skew on a join key. Check the key distribution, confirm AQE skew join settings, and if still slow, salt or isolate the hot key. Re-run and compare stage 7.

INTERVIEW ANGLE

Never answer a slowness question with a config list. Answer with the **method** (locate, classify, fix, verify). Then give examples of fixes per pattern.

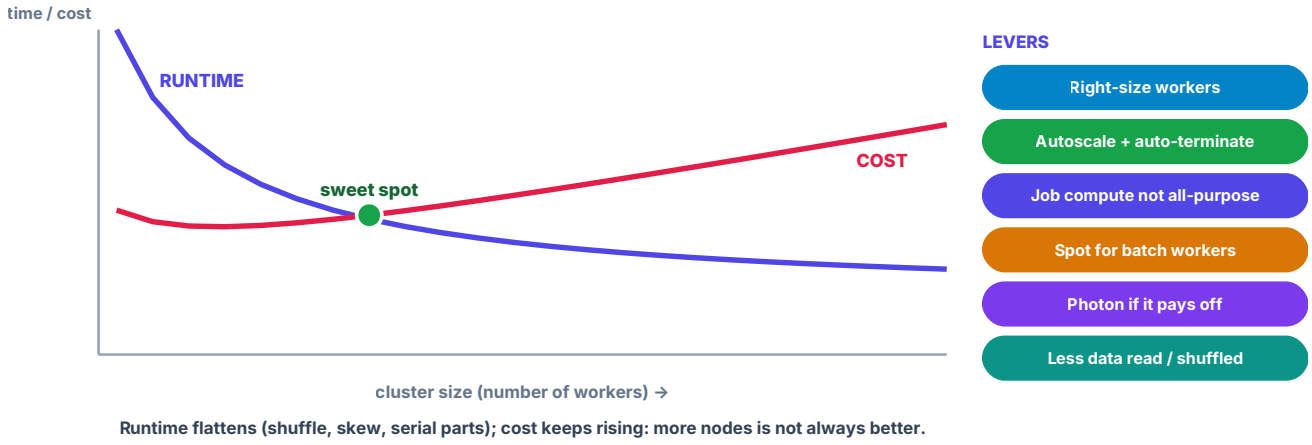
9.5 Cost vs Performance Optimization

★★☆

IN ONE LINE

Meeting the SLA at the lowest total cost, not simply minimising runtime.

Performance work always has a price tag. **Faster is not automatically better**: the goal is to meet the required **SLA** (deadline, freshness) at the **lowest total cost**. Past a point, adding workers yields tiny speed-ups but linear cost.



Runtime flattens (shuffle, skew, serial parts); cost keeps rising: more nodes is not always better.

Cost per run = runtime × (VM price + DBU price) × nodes. A 2× faster job on 3× the nodes costs MORE.
Decide with the business SLA: meet the deadline at the lowest cost, not the lowest runtime at any cost.

Figure 9.5 — Runtime flattens as you scale, cost keeps climbing

Choice	Speed	Cost	When
Fewer, larger jobs vs many small	Similar	Cluster start-up amortised	Batch pipelines
Autoscaling	Adapts	Pay for what is needed	Variable load
Spot instances	Same (risk of loss)	**Much cheaper**	Fault-tolerant batch
Photon	Faster on supported work	Higher DBU rate	Heavy SQL / DataFrame ETL
Optimise the code / layout	**Faster**	**Cheaper** (less compute)	Always the first lever

INTERVIEW ANGLE

The strongest cost optimisation is **doing less work**: prune, filter early, avoid shuffles, compact files. Infrastructure levers come second. Always quote cost **per run** or per TB processed.

9.6 End-to-End Pipeline Optimization

★★★

IN ONE LINE

Apply and validate optimisations across ingestion, transformation, joins and writes, biggest contributor first.

Real optimisation spans the whole pipeline. Treat each stage separately, fix the **largest contributor first**, and validate after every step. Gains compound: removing waste at ingest makes every later stage cheaper.

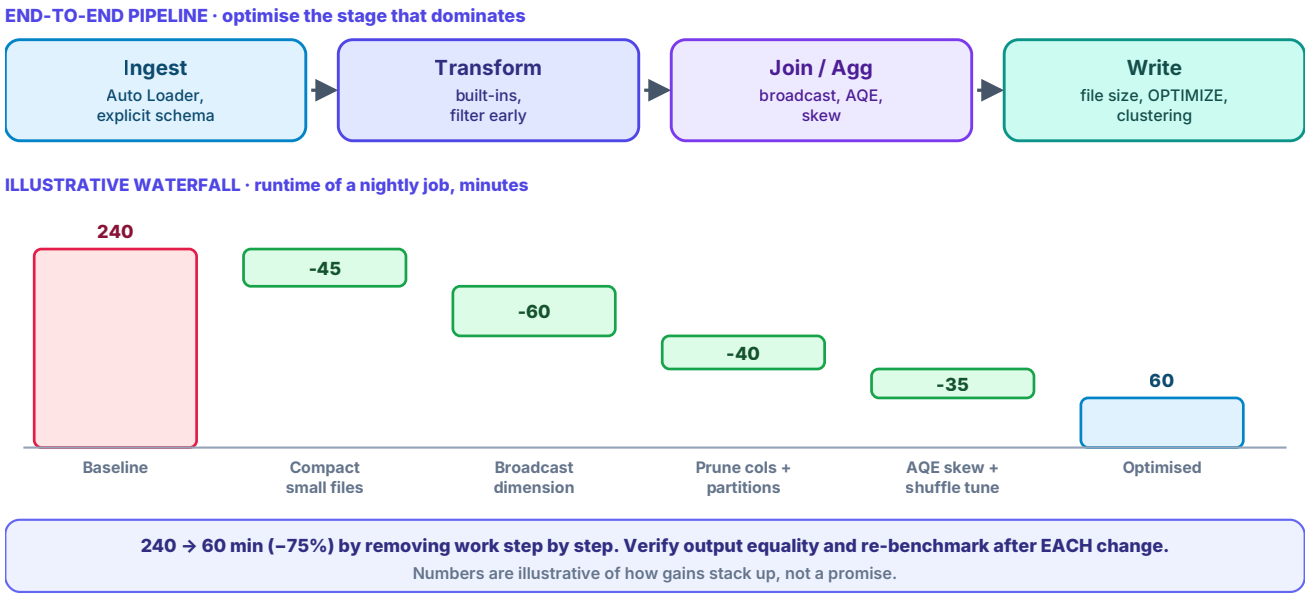


Figure 9.6 — Stage-by-stage tuning, shown as an illustrative waterfall

Stage	Check	Typical fixes
Ingest	File sizes, format, schema inference	Auto Loader, explicit schema, Parquet/Delta, compaction (Parts 5, 8)
Transform	UDFs, wide rows, repeated actions	Built-in functions, early filter / select, cache reused results (3.8, 4.1)
Join / aggregate	Join type, skew, shuffle volume	Broadcast, AQE, pre-aggregate, salting (Part 3, 6)
Write	Output file count and size, partitioning	Right-sized files, OPTIMIZE, clustering, no over-partitioning (2.9, 5.6)
Operate	Compute type, size, cost, monitoring	Job compute, autoscaling, alerts on runtime / data volume (Part 7)

NEVER TRADE CORRECTNESS FOR SPEED

```
# validation: compare optimised output with the baseline
base = spark.table('gold.sales_baseline'); new = spark.table('gold.sales_optimised')
assert base.count() == new.count()
assert base.exceptAll(new).isEmpty() and new.exceptAll(base).isEmpty()
```

INTERVIEW ANGLE

Tell it as a story: *"The nightly job took four hours. The UI showed one join stage at 60% and thousands of small input files. I compacted the files, broadcast the dimension, pruned columns and partitions, and let AQE handle skew: 4 h → 1 h, and I validated output equality."*